

Plytix API: Resources & Documentation

What are the problems are we facing?

Currently, the developer experience at Plytix is split between two distinct environments:

Help Center (Zendesk/Intercom style): Focused on business users; technical info feels like an "afterthought."

API Docs (Swagger/Postman): A functional but "dry" reference that lacks business logic or implementation context. This documentation is not agentic friendly and face several limitations for most of the agentic workflows.

API usage in numbers:

Only a small share of total accounts use the API, but API usage becomes much more relevant when looking at paid accounts.

	Total	Free	Paid
Number of accounts	18202	17355	847
At least 1 API key/password	1632 (8,97%)	1152 (6,64%)	480 (56,67%)

API usage by account plan

	Total	Using API	API usage %
Legacy Paid	786	457	58.14%
Enterprise	9	4	44.44%
Shopify Special	22	7	31.82%
Pro	90	26	28.89%
Standard	3510	238	6.78%
Legacy Free	13785	900	6.53%

Distribution of API credentials per account:

Accounts with 3 or more credentials could be considered API "power users" or accounts with more complex integrations.

Number of credentials	Number of accounts
1 credential	1241
2 credentials	220
3-5 credentials	129
6-10 credentials	38
+11 credentials	4

Commercial value: API as a value indicator

Although API accounts represent a minority of total accounts, they account for a very large share of estimated revenue. This suggests that API usage is strongly associated with higher-value accounts.

Currency	% of accounts using API	% of the revenue
USD	6.20%	55.19%
EUR	14.71%	67.06%



Identified Friction Points:

- **The "Why" vs. "How" Gap:** Users find what an endpoint does, but not how it fits into a larger workflow (e.g., syncing an ERP).
- **Lack of Interactivity:** Developers cannot easily test calls or see real-time results without setting up a local environment.
- **No Centralized "Truth":** If a developer needs to know about Rate Limits and Attribute mapping, they have to search across three different sections.
- **Not AI-first documentation:** Our current documentation is not using OpenAPI format, and the postman collection that is being used is not being rendered by agents. As a result, agents are not able to fetch the information in order to propose the right approach for users.



The "Plytix Hub" Concept (Inspiration)

Ideally to blend the best features of different references into a single experience:

- **From Akeneo:** The clear distinction between "Guides" (narrative) and "Reference" (technical).
- **From inRiver:** The sense of a professional ecosystem where developers feel part of a "Community."
- **From Claude (Anthropic):** The clean, high-performance UI and the "Workbench" feel where testing is intuitive.

Proposed Architecture



1. Getting Started

User orienteed. This is the "Day 0" experience. It should take a developer from 0 to their first API call in under 5 minutes. Topics to be included:

- **Quickstart Guide:** A step-by-step tutorial on API Keys and basic requests.
- **The Plytix Data Model:** A visual guide to how Products, Families, Attributes, and Assets relate to each other.
- **Authentication & Security:** Clear documentation on Bearer tokens and scopes.

|Example:* <https://supabase.com/docs/guides/getting-started>



2. Core Concepts

Educational content that explains the "Plytix Way" of doing things.

- **Pagination & Rate Limits:** Crucial for large catalog-level integrations.
- **Filtering & Sorting:** How to use the Query Language effectively. This is crucial for v3.
- **Key entities:** What are the entities existing in the UI that are available in the API, and what entities are out of the scope of the AP.

|Example: *<https://platform.claude.com/docs/en/build-with-claude/overview>



3. Guides & Tutorials (The "How-to" Library)

Inspirational, solution-based content for the most common use-cases. The approach can be different depending on how we want to communicate. Guides for general solutions (f.e. Connection Plytix with your ERP), Examples for specific use-cases (Integrating Plytix MCP into Claude Custom Connectors) or Tutorials for a step by step process to build any integration.

Some examples as reference:

- **ERP Synchronization:** Best practices for two-way data sync.
- **Bulk Operations:** How to update 50,000 products efficiently without hitting limits.
- **Real-time Updates with Webhooks:** Setting up event-driven architectures.
- **How to integrate Plytix MCP with different LLMs**

|Example:* <https://docs.slack.dev/samples/>



4. Tools

- **SDKs & CLI:** Official wrappers for popular languages and CLI.
- **Changelog:** A technical-only log of API versioning and new features.
- **MCP:** We provide a standardized MCP server implementation. This allows developers to connect Plytix directly to AI-native IDEs (like Cursor, Windsurf, or VS Code) and AI Agents (like Claude or Gemini).

|Examples:*

<https://modelcontextprotocol.io/docs/tools/>

<https://www.saleslayer.com/ai-pim/mcp>



5. API Reference

A modern, interactive version of the current docs. This would include v1/v2 and v3.

- *OpenAPI formatted and code sync:*

The documentation is generated directly from the code's OpenAPI spec to ensure zero "doc-drift" (where the docs say one thing, but the API does another).

- *Optimized for Agentic discovery:*

This is the most critical upgrade. We are optimizing the documentation so that AI Agents (like GitHub Copilot, Cursor, or custom GPTs) can navigate Plytix without human help.

Semantic operationId: Instead of generic IDs like `get_products_123`, we use human-and-machine-readable IDs like `listAllProducts` or `updateProductMedia`.

Descriptions for LLMs: Every endpoint will include a description field written for an LLM's context.

Defined Success/Error Narratives: Instead of just "400 Bad Request," the spec will define common error scenarios in the description, helping an AI agent "self-heal" its code when it encounters a validation error.

x-openai-is-public & Metadata Tags: Using custom OpenAPI extensions to help AI agents categorize which tools are "Read-Only" vs. "Destructive" (Write/Delete).

|Examples:

*<https://developer.squareup.com/reference/square>

<https://docs.stripe.com/api/v2/core/accounts>